01b — Loops: Every Kind, and When to Use Each (beginner-friendly)

A **loop** repeats work so you don't write the same line 100 times. Almost every exam question is "do something to each item" — that's a loop. This file teaches **every kind of loop**, written the clear, explicit way (no shortcuts), with **when to use each one**.

> **Golden habit:** always be able to write the long, explicit loop. The short one-line versions (comprehensions) are just the same loop folded up — we cover them at the end, AFTER you own the explicit form.

The 3 questions that pick your loop

Before writing a loop, ask:

1. **Do I know how many times to repeat?** No → `while`. Yes → `for`.
2. **Am I walking through items (a list/string)?** → `for x in things`.
3. **Do I need the position (index) too, not just the item?** → `enumerate` or `range(len())`.

That's the whole decision. Now each kind in detail.

Loop 1 — `for` over a range of numbers

Use when you want to repeat a fixed number of times, or count through numbers.

```
for i in range(5):        # i takes 0, 1, 2, 3, 4
    print(i)
```

`range` variations (remember: **it stops one BEFORE the end number**):

```
range(5)          # 0 1 2 3 4          (start at 0)
range(2, 6)       # 2 3 4 5            (start..stop-1)
range(1, 10, 2)   # 1 3 5 7 9          (step by 2 — every other numl
range(10, 0, -1)  # 10 9 8 ... 1       (count DOWN with a negative s
```

Scenario: "sum numbers 1 to n", "repeat this 10 times", "count down from 10".

```
total = 0
for i in range(1, n + 1):   # n+1 so n itself is included
    total += i
```

Loop 2 — for over the items of a list or string

Use when you want each **item** and don't care about its position. This is the most common loop on the exam.

```
nums = [3, 1, 4]
for x in nums:            # x is each VALUE: 3, then 1, then 4
    print(x)

for ch in "cat":          # ch is each CHARACTER: 'c', 'a', 't'
    print(ch)
```

Scenario: "add up all the numbers", "check each word", "count the vowels".

```
vowels = 0
for ch in "amalitech":
    if ch in "aeiou":     # is this character a vowel?
        vowels += 1
```

> **Key point:** `for x in nums` hands you the **value**, not its position. If you need the position (to say "the 3rd item" or to change `nums[i]`), use Loop 3 or 4.

Loop 3 — for with `enumerate` (position AND item together)

Use when you need **both** the index and the value.

```
nums = [10, 20, 30]
for i, x in enumerate(nums):   # i = position, x = value
    print(i, x)                # 0 10 , then 1 20 , then 2 30
```

Scenario: "print each item with its position", "find WHERE the biggest value is".

```
best_pos = 0
for i, x in enumerate(nums):
    if x > nums[best_pos]:
        best_pos = i          # remember the POSITION of the best
```

Loop 4 — `for i in range(len(...))` (position, the manual way)

Use when you need the **index** to reach into the list, especially to **change** items or look at **neighbors**.

```
nums = [3, 1, 4]
for i in range(len(nums)):    # i = 0, 1, 2  (every valid position)
    nums[i] = nums[i] * 2     # double each item IN the list
```

Scenario: "modify the list in place", "compare each item to the NEXT one".

```
# compare neighbors — loop to len-1 so nums[i+1] doesn't fall off th
for i in range(len(nums) - 1):
    if nums[i] > nums[i + 1]:
        ...
```

> **enumerate vs range(len()):** both give you the position. Use `enumerate` when you also want the value handed to you; use `range(len())` when you need to index around (`nums[i+1]`, `nums[i-1]`) or write back into the list.

Loop 5 — `while` (repeat until a condition changes)

Use when you **don't know how many times** you'll repeat — you stop when something becomes true.

```
n = 1234
while n > 0:          # keep going as long as n is still positive
    digit = n % 10    # last digit
    n = n // 10       # shrink n — THIS is what eventually ends the
```

Scenario: "peel digits off a number", "keep halving until 1", "repeat until the user is done".

> **Danger — infinite loop:** a `while` only stops when its condition becomes false. If you forget the line that changes it (here, `n = n // 10`), it runs forever. Every `while` must have a line that moves it toward stopping.

Loop 6 — Nested loops (a loop inside a loop)

Use for **grids** (rows × columns) or **every pair** of items. The inner loop runs fully for each single step of the outer loop.

```
grid = [[1, 2, 3],
        [4, 5, 6]]
for r in range(len(grid)):          # outer: each row
    for c in range(len(grid[0])):   # inner: each column in that rou
        print(grid[r][c])           # 1,2,3, then 4,5,6
```

Scenario: "visit every cell in a grid" (Q3), "check every pair of items".

```
# every pair (i, j) with j after i
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            ...
```

> **Speed note:** a loop inside a loop does about `n × n` steps. Fine when n is small (≤ 1000). For big n, use a dict/set instead (file 03).

Loop 7 — Walking two lists together with `zip`

Use when you have **two lists** and want matching items side by side.

```
names  = ["ama", "kofi"]
scores = [90, 80]
for name, score in zip(names, scores):
    print(name, score)     # "ama" 90 , then "kofi" 80
```

Scenario: "pair up names with scores", "compare list A to list B position by position".

`break` and `continue` — controlling any loop

These work inside **any** loop above.

```
for x in nums:
    if x < 0:
        break        # STOP the whole loop right now, exit it
```

```
for x in nums:
    if x % 2 == 0:
        continue     # SKIP the rest of THIS turn, jump to the next
    print(x)         # only runs for odd numbers
```

- **break** = "I'm done, leave the loop." (e.g. found what I was searching for.)
- **continue** = "skip this one, keep looping." (e.g. ignore zeros.)

Scenario for break: "find the first item matching X, then stop." **Scenario for continue:** "process every item except the ones I want to skip."

The 3 things loops do most (memorize these shapes)

Nearly every loop is one of these. The pattern is always: **set up a variable BEFORE the loop, update it INSIDE, use it AFTER.**

A) Running total (sum):

```
total = 0
for x in nums:
    total += x
return total
```

B) Counter (how many match):

```
count = 0
for x in nums:
    if x % 2 == 0:      # your condition
        count += 1
return count
```

C) Best so far (max/min):

```
best = nums[0]
for x in nums:
    if x > best:        # use < for smallest
        best = x
return best
```

These three (sum / count / best) cover most of Q1 and half of Q2. You've already written A and B today (`sum_odd`, `count_digits_divisible`).

Same task, 3 different loops (so you see they're equal)

Task: **sum the numbers in `[10, 20, 30]`.** All three give `60`.

```
# Way 1 — for over items (cleanest here)
total = 0
for x in nums:
    total += x

# Way 2 — for over positions
total = 0
for i in range(len(nums)):
    total += nums[i]

# Way 3 — while with a manual counter
total = 0
i = 0
while i < len(nums):
    total += nums[i]
    i += 1
```

Way 1 is best when you just need the values. Way 2/3 are for when you need the position too. They all do the exact same walking — pick whichever fits what you need from each item.

The one-line version (comprehension) — just a folded loop

Once you're solid with explicit loops, know that Python can fold the simple ones into a line. **This is optional** — the explicit loop is never wrong.

```python
# explicit
squares = []
for x in range(5):
    squares.append(x * x)

# same thing, folded into one line
squares = [x * x for x in range(5)]
```

Read the folded form as: **"give me x*x, for each x in range(5)."** Add `if` to filter:

```python
evens = [x for x in nums if x % 2 == 0]   # keep only evens
```

Use whichever you can read comfortably. On the exam, a correct explicit loop beats a broken one-liner every time.

Quick "which loop?" reference

I want to...	Use
repeat a fixed number of times / count through numbers	`for i in range(...)`
do something to each item (value only)	`for x in things`
get each item AND its position	`for i, x in enumerate(things)`
change items, or look at neighbors (`nums[i+1]`)	`for i in range(len(things))`
repeat until a condition changes (unknown count)	`while condition:`
visit every cell of a grid / every pair	nested loops
walk two lists side by side	`for a, b in zip(list1, list2)`
stop early	`break`
skip one item	`continue`